



科研训练三

1 引言

晏潇 博士

yanxiaosunny@whu.edu.cn



什么是科研？

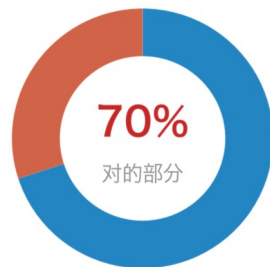
科研 = 发论文



好研究 = 发 **CCF-A** 论文



研究做得好 = 多发 **CCF-A** 论文





怎样的研究算好？

Solve a **Prominent Problem** in a **Principled Way**

- ❖ **Prominent Problem** ：重要问题：问题可能比解法更重要
 - 共性：影响一大类任务，不是某个问题特定 trick
 - 基础：触及算法 / 模型 / 系统中的关键机制——做策略不如做机制
 - 高价值：显著改善重要应用的质量、效率、成本、风险或体验
- ❖ **Principled Way** ：有原则的方法：科研第一性原理是理解
 - 可泛化：不只针对一个问题，可范围，一种基础性思维方式
 - 机理清楚：解释「什么因素 → 通过什么过程 → 导致什么结果」
 - 理论严谨：定义、假设、推导前后一致
 - 证据充分：实验、证明或研究设计有力支持主要主张
- ❖ **Kown by Work, not by Paper Count** ；好的工作，读者读完应该有 takeaway



如何开始第一个科研项目？

重要问题难找？ 解法很难精妙？ 效果有限？

Good Research is to Solve a **Prominent Problem** in a **Principled Way**



Research is to Solve a **Meaningful Problem** in an **Effective Way**

❖ **Meaningful Problem : 有意义的问题**

□ 真实存在的，有实用价值的 vs “填表组合打法”

❖ **Effective Way : 有效的解法**

□ 正确率高的、速度快的、开销低的、使用容易的 vs “限制够多，我就是世界第一”

❖ **先走通流程，熟悉科研的“技术”部分，再提高“艺术部分” vs 非重要工作不做**

❖ **发展眼光看自己的研究，对当下的研究投入十二分热爱**



做科研要很聪明吗？



“气宗”
悟性强

VS



“剑宗”
多锻炼

❖ 科研有一套工作方法，需要在实践学习和熟练

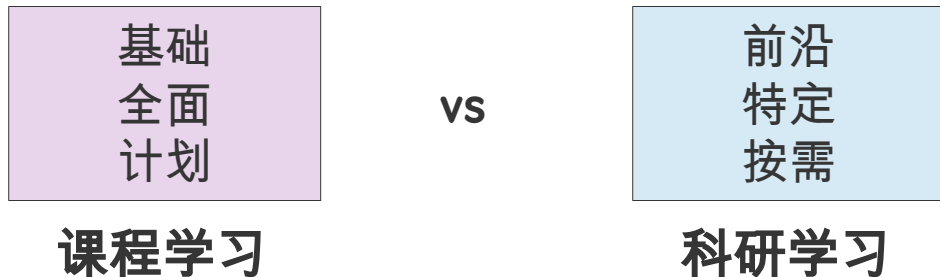
- 文献阅读、实现实验、论文撰写、报告展示
- 起步阶段，挫折和失败非常正常——“论文必中定理”（博士第一定理）
- 持续攀爬学习曲线，熟练之后才能随心应用和创造

❖ “精神小伙”理论



怎样才算准备好科研？

老师，我还没有准备好做研究，计划先系统学习
XXX



❖ 勇于开始

- 结束本科基础课程之后，就可以开始科研 vs 好的研究论文，应该让本科生看懂
- 做中学，目标导向，时常停下规划要学什么
- 乐于发问，通过与人讨论学习
- 科研更看重“动态”智力



科研和工程的区别是什么？

❖ 科研回答「为什么」，形成思路

- 理解规律与机理，从现象中提炼可推广的解释
- 提出可检验的主张，用假设 + 实验 / 证明支持或反驳
- 允许精简到关键机制：20% 的实现验证 80% 的想法

科研 · Research	
核心问题	为什么有效？
产出	新知识 + 可信证据
实现要求	精简到关键机制
成功标准	可发表 · 可复用 · 可复查

❖ 工程解决「稳定性和完整性」，提供服务和产品

- 构建全面稳定的产品，覆盖完整需求与约束
- 处理边界 · 异常 · 兼容性，应对真实环境的多样性
- 追求可维护 · 可部署 · 可运营，在长时间尺度上不崩

工程 · Engineering	
核心问题	能否稳定交付？
产出	可靠交付 + 长期运行
实现要求	完整 · 可维护 · 可部署
成功标准	用户 · SLA · 监控

现实项目在两种思维之间切换：先验证机理，再补齐工程化，二者互补



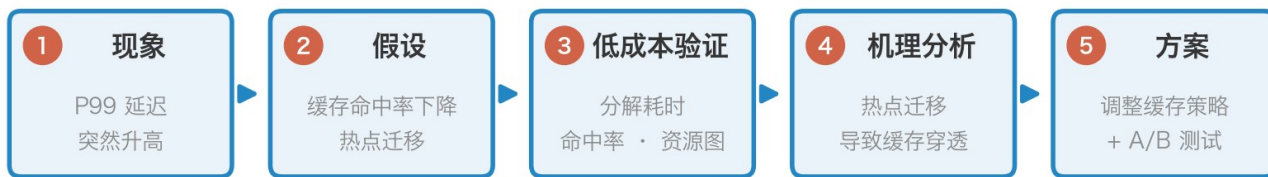
业界也需要科研思维

❖ 案例 · 线上延迟突然升高的排查过程

- 现象：线上延迟 P99 突然升高
- 假设：缓存命中率下降 / 热点迁移
- 低成本验证：分解耗时 · 命中率 · 资源图
- 机理分析：热点迁移导致缓存穿透
- 方案：调整缓存策略并 A/B 测试

❖ “大模型斩杀线”

- 基础的技术能力被模型替代
- 人的价值来自判断与问题定义：好问题 · 边界理解 · 评估设计 · 系统性复盘
- 人才待遇两级分化



与科研范式完全同构

问题 → 机理 → 解法 → 效果



计算机科研如何分类？

❖ 理论 Theory

- 问题：性质 · 界限 · 可解性 | 方法：形式化 · 定理 · 反例 | 证据：证明正确 · 边界紧

❖ 系统 Systems

- 问题：性能 · 可靠性 · 扩展性 | 方法：架构 · 原型 · 测量 | 证据：系统效率 · 故障

❖ 算法 Algorithms

- 问题：建模 · 复杂度 · 近似 | 方法：模型 · 算法 · 实验 | 证据：正确性 · 复杂度

❖ 应用 Applications

- 问题：真实场景 · 数据 · 价值 | 方法：场景 · 评估 · 部署 | 证据：真实任务 · 效果

Graph isomorphism in quasipolynomial time [extended abstract]

Author: László Babai | [Authors Info & Claims](#)

STOC '16: Proceedings of the forty-eighth annual ACM symposium on Theory of Computing • Pages 684 - 697
<https://doi.org/10.1145/2897518.2897542>

理论 · *Graph Isomorphism, Babai 2015*

Efficient Memory Management for Large Language Model Serving with PagedAttention

Authors: Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, Ion Stoica | [Authors Info & Claims](#)

SOSP '23: Proceedings of the 29th Symposium on Operating Systems Principles • Pages 611 - 626
<https://doi.org/10.1145/3600006.3613165>

系统 · *vLLM PagedAttention, SOSP 2023*

Deep Residual Learning for Image Recognition

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun; Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778

算法 · *ResNet, CVPR 2016*

[nature](#) > [articles](#) > article

Article | [Open access](#) | Published: 15 July 2021

Highly accurate protein structure prediction with AlphaFold

应用 · *AlphaFold, Nature 2021*



❖ 关注问题

- 某问题是否可解？最优复杂度是多少？
- 能否证明上界 / 下界？某种不可能性是否成立？
- 近似比能否被突破？

❖ 主要方法

- 形式化定义 · 定理与证明 · 反例构造
- 复杂度分析 · 概率与期望分析 · 归约与不可近似性

❖ 关键证据

- 证明是否正确 · 假设是否必要 · 边界是否紧
- 反例是否覆盖错误主张 · 归约是否有效
- 通常依赖问题的性质和结构

Graph Isomorphism in Quasipolynomial Time

[Extended Abstract]*

László Babai
University of Chicago, USA
laci@cs.uchicago.edu

ABSTRACT

We show that the Graph Isomorphism (GI) problem and the more general problems of String Isomorphism (SI) and Coset Intersection (CI) can be solved in quasipolynomial ($\exp((\log n)^{O(1)})$) time. The best previous bound for GI was $\exp(O(\sqrt{n} \log n))$, where n is the number of vertices (Luks, 1983); for the other two problems, the bound was similar, $\exp(\tilde{O}(\sqrt{n}))$, where n is the size of the permutation domain (Babai, 1983).

Following the approach of Luks's seminal 1980/82 paper, the problem we actually address is SI. This problem takes two strings of length n and a permutation group G of degree n (the "ambient group") as input (G is given by a list of generators) and asks whether or not one of the strings can be transformed into the other by some element of G .

Luks's divide-and-conquer algorithm for SI proceeds by recursion on the ambient group. We build on Luks's framework and attack the obstructions to efficient Luks recurrence via an interplay between local and global symmetry. We construct group theoretic "local certificates" to certify the presence or absence of local symmetry, aggregate the negative certificates to canonical k -ary relations where $k = O(\log n)$, and employ combinatorial canonical partitioning techniques to split the k -ary relational structure for efficient divide-and-conquer. We show that in a well-defined sense, Johnson graphs are the only obstructions to effective canonical partitioning. The central element of the algorithm is the "local certificates" routine which is based on a new group theoretic result, the "Unaffected stabilizers lemma," that allows us to construct global automorphisms out of local information.

* Full paper available at [arXiv:1512.03547](https://arxiv.org/abs/1512.03547). Research supported in part by NSF Grants CCF-7443327 (2014-current), CCF-1017781 (2010-2014), and CCF-0830370 (2008-2010). Any opinions, findings, and conclusions or recommendations expressed in this paper are those of the author and do not necessarily reflect the views of the National Science Foundation (NSF).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

STOC'16, June 19-21, 2016, Cambridge, MA, USA
© 2016 ACM 978-1-4503-4132-5/16/06...\$15.00
<http://dx.doi.org/10.1145/2897518.2897542>

Keywords

Algorithms, complexity of computation, graphs, graph isomorphism, group theory, divide and conquer

Categories and Subject Descriptors

F.2.2 [Nonnumerical algorithms]: Computations on discrete structures—*Graph Isomorphism, Permutation groups*; G.2.2 [Graph Theory]: Graph algorithms—*Automorphism groups, Group theory, Coherent configurations*

1. INTRODUCTION

1.1 Notation, Fraktur, Group Theory Reference

We shall use a number of Fraktur characters. In particular, $\mathfrak{x}$, $\mathfrak{y}$, $\mathfrak{z}$ will denote strings, $\mathfrak{S}_n$ the symmetric group of degree n , $\mathfrak{S}(\Omega)$ the symmetric group acting on the set Ω (the group of all permutations of Ω), $\mathfrak{A}_n$ the alternating group of degree n , and $\mathfrak{A}(\Omega)$ the alternating group acting on the set Ω (the group of even permutations of Ω).

Here is a list of the Roman equivalents of the Fraktur characters we use:

$\mathfrak{x} - \mathfrak{x}$, $\mathfrak{y} - \mathfrak{y}$, $\mathfrak{z} - \mathfrak{z}$, $\mathfrak{A} - \mathfrak{A}$, $\mathfrak{S} - \mathfrak{S}$, $\mathfrak{X} - \mathfrak{X}$, $\mathfrak{y} - \mathfrak{y}$

A permutation group acting on the set Ω is a subgroup $G \leq \mathfrak{S}(\Omega)$. (The " $\leq$ " sign stands for "subgroup.") The degree of G is $|\Omega|$, the size of the permutation domain Ω .

For $x \in \Omega$ and $\sigma \in G$ we write x^σ for the σ -image of x . The set $x^G = \{x^\sigma \mid \sigma \in G\}$ is the G -orbit of x . We say that G is transitive if $x^G = \Omega$ for some (and therefore any) $x \in \Omega$. We note that the orbits of a normal subgroup of a transitive group have equal length. A transitive group $G \leq \mathfrak{S}(\Omega)$ is primitive if $|\Omega| \geq 2$ and there is no nontrivial G -invariant equivalence relation on Ω .

The stabilizer G_x of $x \in \Omega$ is the subgroup $G_x = \{\sigma \in G \mid x^\sigma = x\}$ consisting of those elements of G fixing x . The pointwise stabilizer of a subset $\Delta \subseteq \Omega$ is the subgroup $G_{(\Delta)} = \bigcap \{G_x \mid x \in \Delta\}$. An action of G on a set Δ is a homomorphism $G \rightarrow \mathfrak{S}(\Delta)$.

For basic group theory we refer to the introductory chapters of Rotman [Ro]. Our standard reference for permutation groups is Dixon and Mortimer [DM]. For the polynomial-time algorithmic theory of permutation groups we refer to Seress [Ser]. Permutation groups are given by a list of generators. For a permutation group $G \leq \mathfrak{S}(\Omega)$, the following



系统研究

❖ 系统研究的五步法

- ❑ 工作负载（用户与任务需求）→ 瓶颈分析（资源与关键路径）
- ❑ 架构设计（关键机制与权衡）→ 最小原型（可运行的端到端系统）
- ❑ 测量评估（性能 · 开销 · 可靠性）

❖ 系统研究的目标

- ❑ 更快：公平基线 · 端到端与分阶段耗时
- ❑ 更省：CPU · 内存 · 网络 · 存储 · 能耗
- ❑ 更可靠：故障注入 · 恢复时间 · 失败率
- ❑ 更可扩展：数据 / 节点 / 并发增长下的趋势
- ❑ 通常利用硬件和负载的特性，依赖 Insight

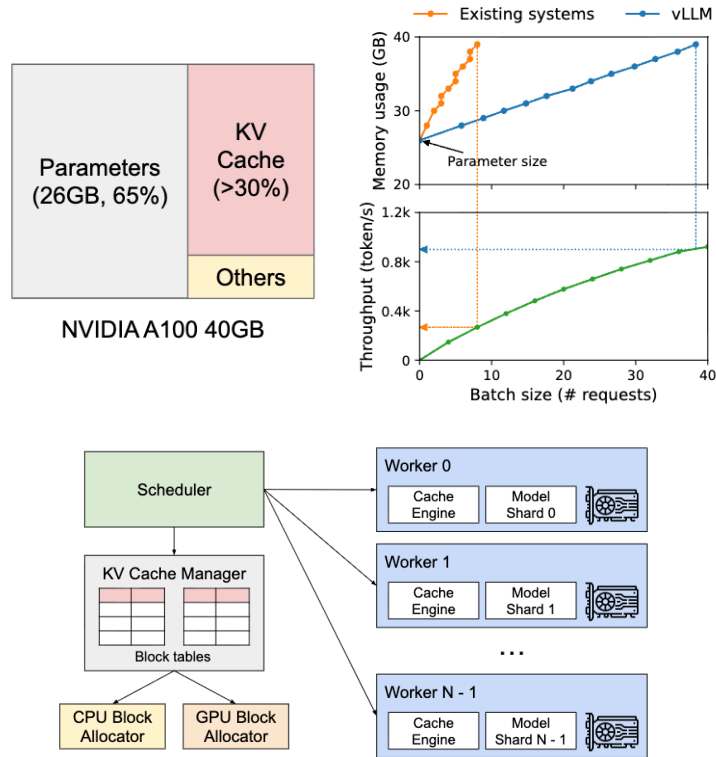


Figure 4. vLLM system overview.



❖ 四个环节

- 问题建模：输入 · 输出 · 优化目标 · 约束
- 算法设计：分解 · 动态规划 · 贪心 · 随机化 · 近似 · 学习
- 分析：正确性 · 复杂度 · 近似比 · 收敛性 · 最坏情况
- 实验：合成 / 真实数据 · 规模趋势 · 对照 · 消融
- 跨越三类：1 偏理论：复杂度 / 近似比 / 收敛性证明；2 偏系统：在真实规模硬件上验证实现效率与扩展性；3 偏应用：解决推荐 / 调度 / 路径 / 压缩等真实任务
- 也依赖对问题的深入理解

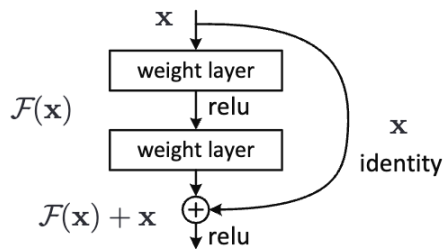
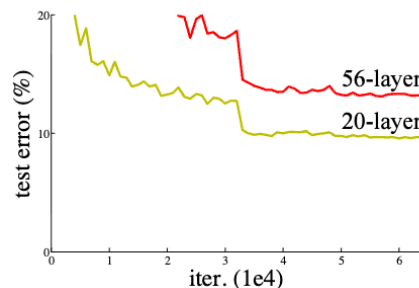
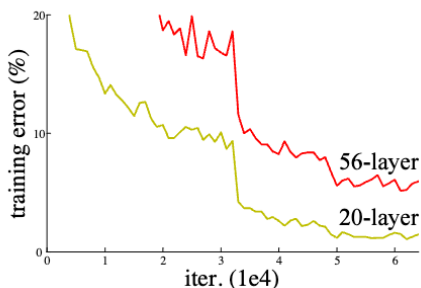


Figure 2. Residual learning: a building block.





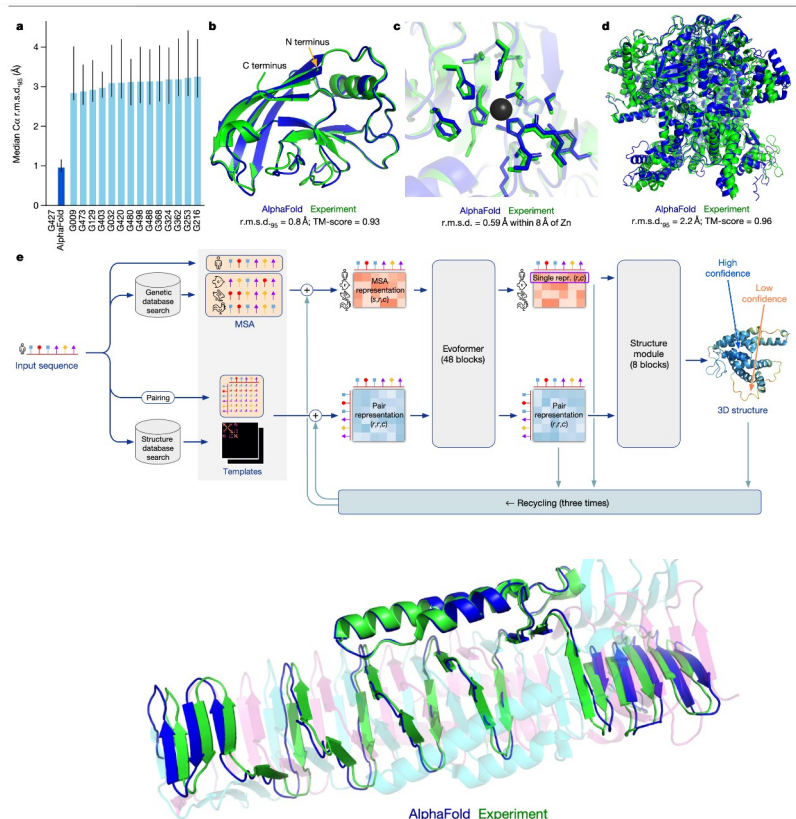
应用研究

❖ 五个核心问题

- ❑ 真实场景（问题在哪发生？频率？） · 目标用户（谁受影响？痛点？）
- ❑ 可获得数据（代表什么？是否有偏差？） · 实际约束（隐私 · 成本 · 延迟）
- ❑ 评价指标（什么才算改善？与真实目标对齐吗？）

❖ 应用价值判定三要素

- ❑ 真实任务：问题在生产 / 临床 / 教育 / 金融中存在，而不是合成
- ❑ 数据代表性：训练 / 评测分布对齐部署分布；指标对齐
- ❑ 离线指标 $\neq$ 线上指标时，回到真实目标重新设计
- ❑ 关键在于理解应用领域和结合计算机知识





硕士与博士的培养是怎样的？

❖ 硕士：对一个领域有较深入了解，解决某个问题

- 掌握领域基本文献与方法 · 在指导下完成完整研究流程
- 能够独立解决一个明确问题 · 形成专业能力或研究成果
- 半年课程 + 熟悉领域；两年研究，产出 1-3 个工作；半年找工作和毕业论文

❖ 博士：对一个领域有非常深入了解，持续解决一类问题

- 识别重要且未解决的问题 · 形成独立研究判断与方向
- 能够持续解决一类问题 · 扩展领域知识边界
- 半年课程 + 熟悉领域；四年研究，产出 3 个以上工作；半年找工作和毕业论文
- 工作体系性和亮点工作比工作数量更重要



科研项目的大致流程是什么？

❖ ① 定义问题：价值 · 边界

❑ 问题真实？值得解决？范围清楚？

❖ ② 调研相关工作：共识 · 冲突 · 空白

❑ 已有工作做到哪？关键假设？证据覆盖？

❖ ③ 设计解决方案：机理 · 假设 · 权衡

❑ 关键机理？方案如何作用？失败模式？

❖ ④ 实现与实验：快速迭代 · 证据

❑ 基线公平？变量清晰？负结果记录？

❖ ⑤ 论文与交流：主张 · 局限 · 修订

❑ 主张明确？证据对应？贡献与边界？

❖ 过程不一定线性，通常来回反复

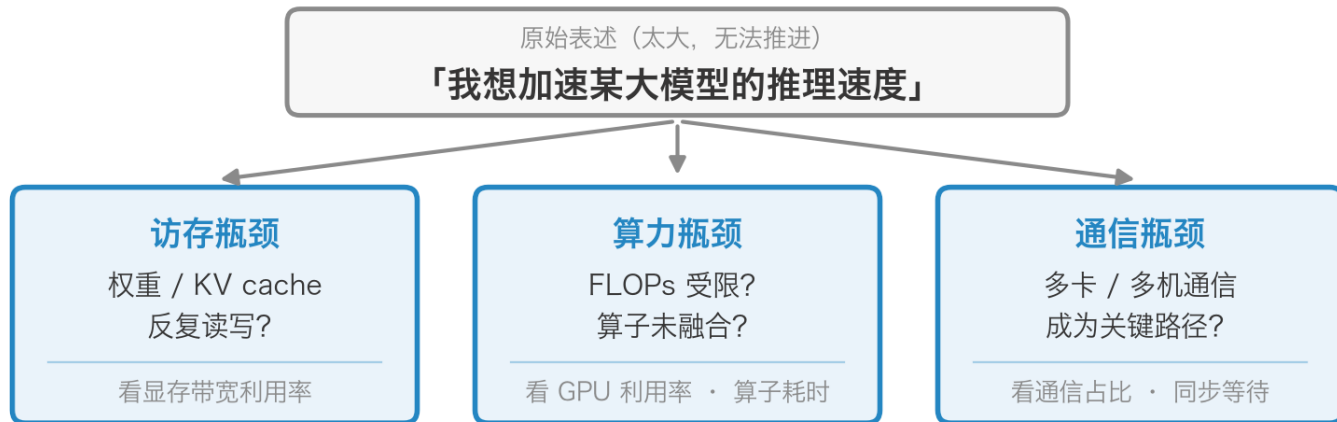




① 定义问题

❖ 定义问题的四个检查

- ❑ 真实存在：案例 · 数据 · 用户 · 系统记录
- ❑ 范围清楚：对象 · 场景 · 目标 · 非目标都明确
- ❑ 可推进：通过分析 · 实现 · 实验能产生新证据 vs “可验证性”限制了科研，Benchmark
- ❑ 成功可观察：用什么指标判断问题真的被解决了





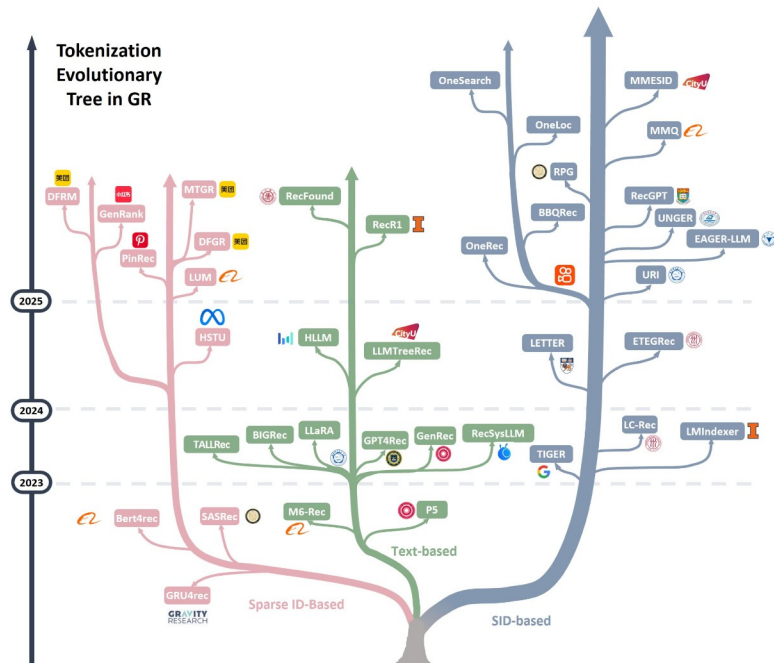
② 调研相关工作

❖ 调研的四个核心问题

- ❑ 解决到什么程度：已有工作能否在主要指标上达到要求？
- ❑ 依赖哪些关键假设：放在我的场景还成立吗？
- ❑ 证据覆盖哪些场景与边界：哪些场景没测？
- ❑ 共识 · 冲突 · 空白：哪里有争议？哪里没人做？

❖ 工具与方法

- ❑ 检索（关键词 · 同义词 · 学者 · 会议专题）→ 筛选（标题 → 摘要 → 结论 → 实验）
- ❑ 三遍阅读（鸟瞰 → 精读 → 复现式阅读）→ 做表比较（问题 · 方法 · 假设 · 指标）
- ❑ 问题发现：把比较表中的空白转成研究问题





③ 设计解决方案

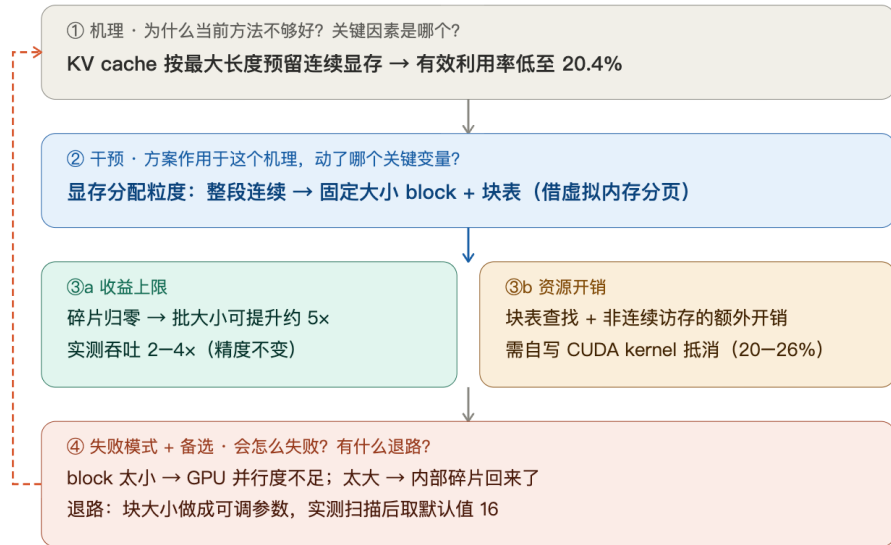
❖ 方案设计的四步走

- ❑ 机理：为什么当前方法不够好？关键因素是哪个？
- ❑ 干预：方案如何作用于这个机理？动了哪个关键变量？
- ❑ 收益上限 + 资源开销：理论上最好能好多少？要花多少算力 / 内存 / 时间？
- ❑ 失败模式 + 备选：会怎么失败？有什么退路？

❖ 设计阶段最容易踩的坑

- ❑ 只换模型 / 只换超参 / 只换数据 → 没有指出机理，就不能称为「设计」
- ❑ 把「指标涨了 1 个点」当成「设计成功」→ 必须解释为什么涨、还能不能涨

案例：LLM 推理服务吞吐上不去



失败模式反过来会修正对机理的理解 —— 设计是循环，不是一条直线

数据来源：Kwon et al., PagedAttention / vLLM, SOSP 2023



④ 实现与实验

❖ 先跑通公平可重复的基线

- 一致的硬件 / 软件栈 / 数据划分，记录环境 · 参数 · 随机种子 · 数据指纹
- 明确基线与你方法的「唯一区别」，对基线做相同强度的调参与消融

❖ 一次只加一个关键机制

- 从最简单的 baseline 开始，每次只引入一个新变量
- 用消融实验证明「哪个机制真的有用」

❖ 从 Claim 反推对照 · 变量 · 指标

- 要证明主张，需要哪些对照实验？哪些变量必须控制？哪些指标是关键证据？

❖ 负结果不是失败，是研究记录

- 没跑通的实验可能说明假设错了 / 边界条件没考虑 / 实现有 bug

❖ 最终实验设计要完整全面



⑤ 论文撰写与交流

❖ 论文结构与写作

- 摘要—引言—背景—问题定义—解法—实验—相关工作—总结
- 要以“理解”贯穿全文：问题的核心是什么？已有方法为什么不好？我们为什么好？

❖ 写作会暴露的问题

- 主张与证据不对齐 · 贡献说不清楚
- 问题定义含糊 · 实验设计有漏洞

❖ 论文投递与审稿

❖ 可复现性与学术会议

- 论文：公开记录 + 同行评审
- 代码与数据：让别人能跑出来
- 实验日志：记录每次改动与判断
- 参加学术会议，多听和认识顶尖学者非常重要



用第一性原理指导科研

第一性原理（First Principles）是一种从最基础、不可再分的事实出发进行推理的思维方式，核心是**“回到事物的本质”**，而不是依赖类比和惯例

❖ 科研的第一性原理是“理解”

- 问题相关的核心概念是哪些？它们如何相互影响？
- 已有工作的缺陷，是由于什么原因造成的？
- 我们的解法，为什么效果良好？
- 对不同场景和数据集的性能差异，如何解释？
- 用同一概念体系，把这些内容理清楚

❖ 深入才能浅出

- 真正理解：简单直观解释相关工作、研究问题和解法思路：15 秒，1 分钟，5 分钟，30 分钟
- 多练习对话和讲，多与人探讨：贝曼学习法



理清“标准”和参照“样本”

❖ 标准和目标：怎样才算做得好？明确标准才能找到努力方向

- ❑ 论文调研：找到核心和最近论文，理清核心概念和发展脉络，总结出可能研究方向
- ❑ 论文写作：Sparking，Smooth，Steady（来自“防御性写作”）
- ❑ 画图：易读且美观
- ❑ 审稿回复：直接且翔实
- ❑ 简历和申请文书：突出自己优点，响应读者需求

❖ 样本：找到优秀的参照

- ❑ 样本帮助理解标准
- ❑ 每个小领域有自己的标准和优秀样本
- ❑ 认真研读和理解样本

❖ 课程尝试为论文调研、方法构思、实验设计、论文写作、文书写作等提供样本



科研新趋势

❖ 问题 · 场景 · 数据可能正在变得比解法更重要

- 企业作为问题 · 场景 · 数据的拥有者，影响力在变强
- 实践和实际更加重要
- “差异化” vs “打不过就加入”

❖ 论文通胀，工具 · 框架 · 产品变得更重要

- AI“上三会”2025年总录用数超1.2万篇
- 实际可用的成果更加吸引人

❖ “大模型斩杀线”，人才要求变高，待遇两级分化

❖ 论文评价中性化，写作能力反而更加重要（“防御性写作”体系）

- 大模型普及，论文写作质量变高，大多数论文能拿到中性评价
- 审稿人素质参差不齐，倾向于中性评价
- 审稿回复成为中稿重要因素，“对线”能力至关重要，这也要求深入理解



科研诚信

❖ 显性科研诚信问题

- ❑ 数据和实验结果造假
- ❑ 抄袭剽窃（有时并不是故意）
- ❑ 违规使用大模型，编造参考文献
- ❑ 一稿多投
- ❑ 串通审稿

❖ 隐性科研诚信问题

- ❑ Cherry Picking，选择性报告实验结果
- ❑ 不报告重要基线或对基线不进行充分调优
- ❑ 违规署名（“友好”挂名）

❖ 诚信是底线，一次欺骗，终身污点，多年努力和信誉归零

❖ 洁身自好，拒绝不诚信行为，勇于和不诚信行为做斗争



研究经历分享—刘仁杰，MIT 博士一年级

❖ 基础信息

- 南方科技大学 2019 级本科生，2023 级硕士生
- 研究方向：机器学习系统，专注系统框架和执行效率提升
- 具体研究：图学习系统、大模型推理、图检索增强，智能体系统

❖ 成果奖项

- 发表 CCF-A 类论文 7 篇，其中第一作者（含共同）2 篇
- 数据库旗舰 SIGMOD 等 3 篇，系统旗舰 SOSP 等 2 篇，机器学习旗舰 ICML 等 2 篇
- 2024 年国家奖学金

❖ 科研经历

- 亚马逊上海 AI 研究院：图学习训练系统，GPU、分布式、SSD 系统框架和执行加速
- 纽约大学：图检索增强，数据集构建，图检索算法；智能体系统，网页智能体执行范式
- 南方科技大学：大模型推理优化，KV Cache 压缩，V-O 位置编码

❖ 博士申请

- 获得 MIT、UC Berkeley、CMU CS PhD Offer

